

মাল্টি-জিপিইউ ট্রেনিং প্যারাডাইম (**Multi-GPU Training Paradigms**) বিষয়টি অত্যন্ত গুরুত্বপূর্ণ, বিশেষ করে যখন আমরা বড় বড় AI মডেল (যেমন ChatGPT বা লার্জ ল্যাঙ্গুয়েজ মডেল) ট্রেন করতে চাই।

সহজ কথায় বলতে গেলে, যখন একটি মডেল বা ডাটা এত বড় হয়ে যায় যে একটিমাত্র জিপিইউ (GPU)-তে তা জায়গা হয় না বা অনেক সময় লাগে, তখন আমরা একাধিক জিপিইউ ব্যবহার করি। এই জিপিইউগুলো একে অপরের সাথে কীভাবে কাজ করবে, তার পদ্ধতিগুলোকেই প্যারাডাইম বলা হয়।

নিচে বিস্তারিত এবং সহজভাবে প্রতিটি পদ্ধতি আলোচনা করা হলো:

১. মডেল প্যারালেলিজম (Model Parallelism)

এটি সবচেয়ে সহজবোধ্য পদ্ধতি। মনে করুন আপনার একটি মডেল আছে যাতে ১০টি লেয়ার আছে। কিন্তু আপনার জিপিইউর মেমোরি ছোট, তাই পুরো ১০টি লেয়ার এক জিপিইউতে ধরছে না।

- পদ্ধতি: আপনি প্রথম ৫টি লেয়ার রাখলেন **GPU-1** এ এবং পরের ৫টি লেয়ার রাখলেন **GPU-2** এ।
- কাজ করার ধরন: ডাটা প্রথমে GPU-1 এ প্রসেস হবে, তারপর সেই রেজাল্ট GPU-2 তে পাঠানো হবে।
- সুবিধা: বড় মডেল ট্রেন করা সম্ভব যা একটি জিপিইউতে ধরে না।
- অসুবিধা: এখানে একটি বটলনেক (**Bottleneck**) তৈরি হয়। যখন GPU-1 কাজ করে, তখন GPU-2 বসে থাকে। আবার GPU-2 যখন কাজ করে, GPU-1 বসে থাকে। একে বলা হয় অলস সময় (**Idle time**)।

২. ডাটা প্যারালেলিজম (Data Parallelism)

এটি বর্তমানে সবচেয়ে বেশি ব্যবহৃত পদ্ধতি। এখানে মডেলটি ছোট, কিন্তু ডাটা অনেক বেশি।

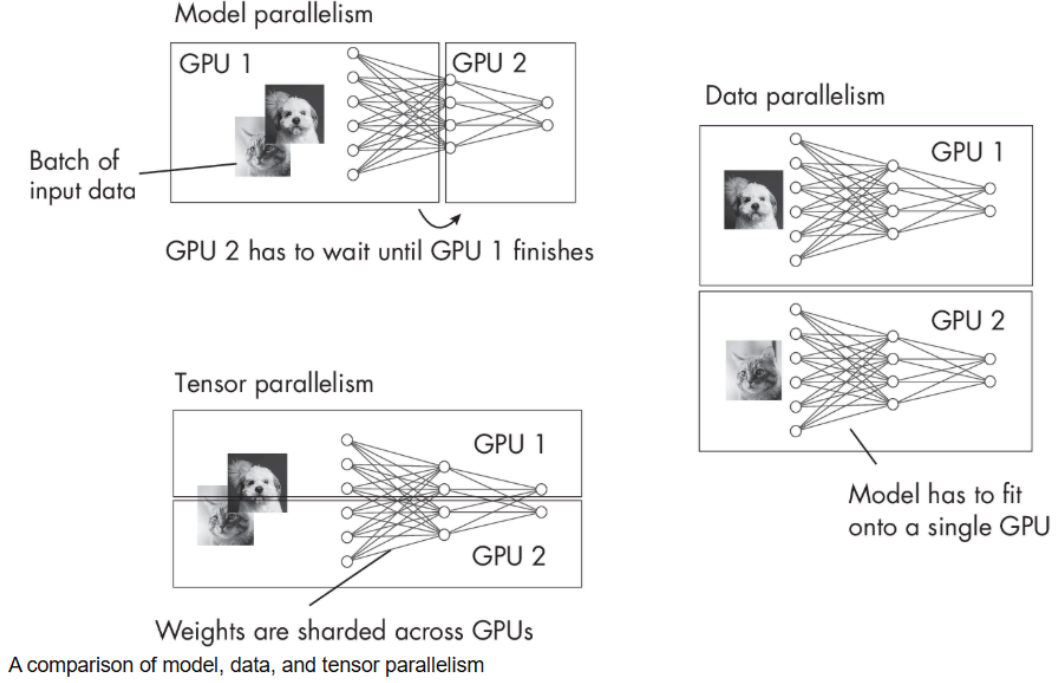
- পদ্ধতি: আপনার কাছে যে কটি জিপিইউ আছে, সবগুলোতে মডেলের একটি করে হুবহু কপি (**Copy**) রাখা হয়। এরপর আপনার বিশাল ডাটা বা ব্যাচকে ছোট ছোট ভাগে (Microbatches) ভাগ করে প্রতিটি জিপিইউতে পাঠিয়ে দেওয়া হয়।
- কাজ করার ধরন: প্রতিটি জিপিইউ আলাদা আলাদা ডাটা নিয়ে কাজ করে এবং গ্রেডিয়েন্ট (Gradients) বের করে। শেষে সব জিপিইউর রেজাল্ট গড় (Average) করে মাইন মডেল আপডেট করা হয়।
- সুবিধা: জিপিইউগুলো একসাথে সমান্তরালে (Parallel) কাজ করতে পারে, তাই সময় অনেক কম লাগে।
- অসুবিধা: প্রতিটি জিপিইউতে পুরো মডেলটি রাখার মতো মেমোরি (VRAM) থাকতে হবে। মডেল অনেক বড় হলে এটি আর কাজ করে না।

৩. টেনসর প্যারালেলিজম (Tensor Parallelism)

এটি মডেল প্যারালেলিজমের একটি উন্নত রূপ। এখানে আমরা শুধু লেয়ার ভাগ করি না, বরং লেয়ারের ভেতরের ম্যাট্রিক্স বা টেনসরকে ভাগ করে ফেলি।

- পদ্ধতি: ধরুন একটি বড় ম্যাট্রিক্স মাল্টিপ্লিকেশন করতে হবে। আপনি ম্যাট্রিক্সটিকে দুই ভাগ করে অর্ধেক দিলেন GPU-1 কে আর অর্ধেক GPU-2 কে। (চিত্র ৭.১ এ যেমনটি দেখানো হয়েছে)।
- কাজ করার ধরন: জিপিইউগুলো ম্যাট্রিক্সের আলাদা আলাদা অংশ একই সাথে ক্যালকুলেশন করে।
- সুবিধা: এটি মেমোরির সমস্যার সমাধান করে এবং ডাটা প্যারালেলিজমের মতো দ্রুত কাজ করতে পারে।

- অসুবিধা: জিপিইউগুলোর মধ্যে খুব দ্রুত ডাটা আদান-প্রদান (Communication) করতে হয়। ইন্টারনেটের স্পিড বা ব্যান্ডউইথ কম হলে এটি ধীরগতি হয়ে যায়।



- মডেল প্যারালেলিজম: লেয়ার ভাগ করা হয় (Layer-wise)।
- ডাটা প্যারালেলিজম: ডাটা ভাগ করা হয় (Data-wise)।
- টেনসর প্যারালেলিজম: অপারেশন বা ম্যাট্রিক্স ভাগ করা হয় (Intra-op)।

8. পাইপলাইন প্যারালেলিজম (Pipeline Parallelism)

এটি মডেল প্যারালেলিজমের একটি স্মার্ট ভার্শন।

- পদ্ধতি: এখানেও লেয়ার ভাগ করা হয়, তবে জিপিইউগুলোকে বসিয়ে রাখা হয় না। ডাটাগুলোকে ছোট ছোট টুকরো করে একটির পর একটি পাঠিয়ে পাইপলাইনের মতো কাজ চালানো হয়।
- কাজ করার ধরন: যখন GPU-2 প্রথম টুকরো ডাটা নিয়ে কাজ করছে, তখন GPU-1 বসে না থেকে দ্বিতীয় টুকরো ডাটা প্রসেস করা শুরু করে।
- সুবিধা: মডেল প্যারালেলিজমের চেয়ে অনেক বেশি দক্ষ।
- অসুবিধা: এটি সেটআপ করা বেশ জটিল এবং মাঝে মাঝে কিছু "Idle Bubbles" বা খালি সময় তৈরি হয়।

৫. সিকোয়েন্স প্যারালেলিজম (Sequence Parallelism)

এটি মূলত ট্রান্সফরমার বা LLM-এর জন্য তৈরি। ট্রান্সফরমার মডেলে যদি ইনপুট সেন্টেন্স অনেক বড় হয়, তবে মেমোরি অনেক বেশি লাগে।

- পদ্ধতি: এখানে ইনপুট সিকোয়েন্স বা বড় বাক্যটিকে টুকরো টুকরো করে বিভিন্ন জিপিইউতে ভাগ করে দেওয়া হয়। (চিত্র ৭.৩ দেখুন)।
- সুবিধা: অনেক লম্বা টেক্সট বা বড় ইনপুট প্রসেস করতে সাহায্য করে।
- অসুবিধা: বাক্য ভেঙে ফেলার কারণে মডেলের নিখুঁতভাবে বোঝার ক্ষমতা (Accuracy) কিছুটা কমতে পারে।

Recommendations

মডেলের আকার	কি ব্যবহার করবেন?
ছোট মডেল (১টি জিপিইউতে ধরে)	Data Parallelism (এটিই সবচেয়ে সহজ এবং দ্রুত)।
বড় মডেল (১টি জিপিইউতে ধরে না)	Tensor Parallelism অথবা Model Parallelism ।
আধুনিক বড় মডেল (LLM)	সাধারণত Data Parallelism + Tensor Parallelism এর একটি হাইব্রিড মিশ্রণ ব্যবহার করা হয়।